

SPRING BOOT TESTING

Unit Testing, Mockito, Patrón AAA y Principios SOLID

Guía práctica basada en el proyecto **unit-testing**: cómo pensar, escribir y entender pruebas en una aplicación Spring Boot.

Proyecto: Empresa API con Controller, Service, Repository y tests.

Enfoque: explicación con analogías, cuadros, ejemplos y lectura de código.

Objetivo: que puedas leer un test y saber qué historia está probando.

Mapa De La Guía

1. Arquitectura Spring Boot

Controller, Service, Repository y cómo se comunican.

2. Unit Testing

Qué es una unidad y qué NO debería probar un unit test.

3. Mockito

Mocks, stubs, `when`, `thenReturn`, `@Mock` e `@InjectMocks`.

4. Patrón AAA

Arrange, Act, Assert aplicado a tus tests de Empresa.

5. Tests De Controller

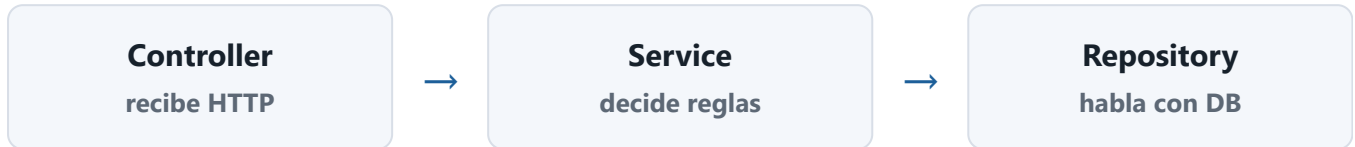
`@WebMvcTest`, `MockMvc`, JSON y endpoints.

6. SOLID

Los cinco principios explicados con ejemplos Spring Boot.

1. La App Spring Boot Como Un Restaurante

Una aplicación Spring Boot suele dividirse en capas. Esa división ayuda a que cada parte tenga una responsabilidad clara. En tu proyecto de Empresa se ve así:



Analogía: piensa en un restaurante. El **Controller** es el mozo que recibe el pedido, el **Service** es el chef que decide cómo prepararlo, y el **Repository** es la despensa donde se guardan o buscan los ingredientes.

Capa	En tu proyecto	Responsabilidad	Qué se prueba
Controller	<code>EmpresaController</code>	Recibir requests HTTP y devolver responses.	Rutas, status HTTP, estructura JSON.
Service	<code>EmpresaServiceImpl</code>	Aplicar reglas de negocio.	Decisiones: crear, actualizar, listar, eliminar.
Repository	<code>EmpresaRepository</code>	Consultar o guardar datos.	En unit test normalmente se reemplaza por un mock.

Flujo Real Al Crear Empresa

1. El usuario manda un `POST /empresa/v1/`.
2. El `EmpresaController` recibe el `EmpresaRequest`.
3. El controller llama a `empresaService.crear(request)`.
4. El service pregunta al repository si existe el documento.
5. Si existe, responde conflicto. Si no existe, guarda y responde OK/CREATED.

2. Qué Es Unit Testing

Un **unit test** prueba una unidad pequeña del sistema. No intenta levantar todo el mundo. Su misión es responder: "si esta pieza recibe estas condiciones, ¿devuelve el resultado esperado?".

Unit test bueno

Prueba una clase o método con dependencias controladas.

Ejemplo: probar

`EmpresaServiceImpl.crear()` usando un `EmpresaRepository` falso.

Unit test confuso

Depende de base de datos, red, servidores externos o datos cambiantes.

Si falla, no sabes si falló tu lógica, PostgreSQL, Docker o la configuración.

Analogía: si quieres probar si una licuadora funciona, no necesitas abrir un restaurante completo. Pones fruta, presionas el botón y miras si salió jugo. Eso es unit testing: aislar lo importante.

Qué Está Probando Tu Test De Service

En `EmpresaServiceImplTest` estás probando reglas como:

- Si la empresa ya existe, no debe guardarse otra vez.
- Si la empresa no existe, debe guardarse correctamente.
- Si busco un ID existente, debe retornar la empresa.
- Si busco un ID inexistente, debe retornar "No hay datos".
- Si actualizo una empresa inexistente, debe responder que no existe.
- Si elimino una empresa existente, debe hacer eliminación lógica.

Idea clave: no estás probando Hibernate ni PostgreSQL. Estás probando las decisiones de tu clase de servicio.

3. Mockito: Actores Para Tu Prueba

Mockito sirve para crear objetos falsos que simulan comportamientos. Estos objetos se llaman **mocks**. En vez de usar una base de datos real, haces que el repository responda exactamente lo que necesitas para cada escenario.

Analogía: Mockito es como contratar actores para una escena. El service es real, pero el repository es un actor que dice las líneas que tú le escribes.

Mockito	Qué significa	Ejemplo mental
<code>@Mock</code>	Crea una dependencia falsa.	Una despensa falsa.
<code>@InjectMocks</code>	Crea la clase real e inyecta los mocks dentro.	Chef real usando despensa falsa.
<code>when(...).thenReturn(...)</code>	Define qué debe responder el mock.	"Si te preguntan X, responde Y".
<code>any()</code>	Acepta cualquier argumento.	"No importa qué empresa llegue".
<code>anyString()</code>	Acepta cualquier texto.	"No importa qué número de documento llegue".

Ejemplo De Tu Proyecto

```
@Mock
private EmpresaRepository empresaRepository;

@InjectMocks
private EmpresaServiceImpl empresaService;
```

Esto significa: "quiero probar el `EmpresaServiceImpl` real, pero no quiero una base de datos real. Métele un `EmpresaRepository` falso".

```
when(empresaRepository.existsByNumeroDocumento(anyString()))
    .thenReturn(true);
```

Traducción: "cuando el service pregunte si existe una empresa por documento, responde que sí".

4. Patrón AAA

AAA es una forma simple y ordenada de escribir tests.

Arrange

Preparar el escenario.

Datos falsos, mocks y respuestas esperadas.

Act

Ejecutar la acción.

Llamar al método real que quieres probar.

Assert

Verificar el resultado.

Comparar lo obtenido con lo esperado.

Analogía: quieres probar una alarma. Arrange: cierras la puerta y activas la alarma. Act: abres la puerta. Assert: verificas que sonó.

AAA En Tu Test “Empresa Ya Existe”

```
@Test
displayName("Deberia retornar conflicto si la empresa ya existe")
void testDeberiaRetornarEmpresaExiste() {
    // Arrange
    when(empresaRepository.existsByNumeroDocumento(anyString()))
    .thenReturn(true)

    // Act
    ResponseEntity<BaseResponse<Empresa>> resultado =
        empresaService.crear(empresaRequest);

    // Assert
    assertNotNull(resultado);
    assertEquals(Constants.CODE_EXIST, resultado.getBody().getCode());
    assertFalse(resultado.getBody().getObjeto().isPresent());
}
```

Parte	Línea mental	Qué demuestra
Arrange	El repository dice que la empresa ya existe.	Preparas el caso negativo.
Act	Llamas a <code>empresaService.crear()</code> .	Ejecutas la lógica real.
Assert	Esperas código de existencia y objeto vacío.	Confirmas que el service tomó la decisión correcta.

Regla práctica: cada test debe contar una historia clara. Si necesitas explicar demasiado el test, quizá está probando muchas cosas a la vez.

5. Tests De Controller Con MockMvc

En `EmpresaControllerTest` ya no estás probando la lógica completa del service. Estás probando la capa web: rutas, JSON, status HTTP y formato de respuesta.

```
@WebMvcTest
class EmpresaControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private EmpresaService empresaService;
}
```

`@WebMvcTest` levanta una parte pequeña de Spring enfocada en MVC. El controller es real, pero el service se reemplaza por un mock.

Analogía: aquí no estás probando al chef. Estás probando al mozo: si recibe un pedido por la ruta correcta, si lo manda al service y si entrega una respuesta con buen formato.

Ejemplo: POST Para Registrar Empresa

```
mockMvc.perform(MockMvcRequestBuilders.post("/empresa/v1/")
    .contentType(MediaType.APPLICATION_JSON)
    .content(ObjectMapper.writeValueAsString(empresaRequest)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.code").value(2001))
    .andExpect(jsonPath("$.objeto.numeroDocumento").value("123456789"));
```

Elemento	Qué hace
<code>MockMvc</code>	Simula requests HTTP sin abrir un servidor real.
<code>ObjectMapper</code>	Convierte el objeto Java a JSON.
<code>jsonPath("\$.code")</code>	Busca el campo <code>code</code> dentro del JSON de respuesta.
<code>@MockBean</code>	Reemplaza un bean de Spring por un mock dentro del contexto de test.

6. Tipos De Pruebas En Spring Boot

Tipo	Qué prueba	Herramientas comunes	Cuándo usarlo
Unit test	Una clase aislada.	JUnit, Mockito.	Reglas de negocio del service.
Web MVC test	Controller y capa web.	<code>@WebMvcTest</code> , <code>MockMvc</code> .	Rutas, JSON, status.
Integration test	Varias capas juntas.	<code>@SpringBootTest</code> , DB real o Testcontainers.	Validar que todo conecta bien.
Repository test	Consultas a base de datos.	<code>@DataJpaTest</code> .	Probar queries, mappings y JPA.

Importante: no todo test debe levantar Spring completo. Mientras más grande el contexto, más lento y más difícil de diagnosticar será el fallo.

7. Principios SOLID En Spring Boot

SOLID son cinco principios para diseñar código más mantenible. No son reglas mágicas, pero sí ayudan a que tu proyecto sea más fácil de probar, cambiar y entender.

Analogía general: SOLID es como organizar una cocina profesional. Cada persona tiene una tarea, las herramientas tienen un uso claro, y puedes cambiar una pieza sin destruir todo el restaurante.

S: Single Responsibility Principle

Una clase debe tener una sola razón para cambiar.

En Spring Boot, el controller no debería contener reglas de negocio ni SQL. El service no debería saber de HTTP puro. El repository no debería decidir mensajes de respuesta.

Mal enfoque	Buen enfoque
El controller valida duplicados, guarda en DB y arma toda la lógica.	El controller recibe request y delega al service.
El service arma rutas HTTP.	El service aplica reglas: existe, no existe, guardar, actualizar.

En tu proyecto, `EmpresaController` delega en `EmpresaService`. Eso se acerca bien a SRP.

O: Open/Closed Principle

El código debería estar abierto a extensión, pero cerrado a modificación.

Significa que si mañana agregas una nueva forma de validar empresas, idealmente no deberías romper todo lo existente.

Si tienes una cafetera modular, puedes agregar un filtro nuevo sin reconstruir la cocina.

En Spring Boot esto se logra usando interfaces, clases pequeñas y estrategias. Por ejemplo:

```
public interface EmpresaValidator {  
    void validar(EmpresaRequest request);  
}
```

Luego podrías tener `DocumentoValidator`, `DireccionValidator` o `RucValidator` sin meter todo dentro de un único método enorme.

L: Liskov Substitution Principle

Una implementación debe poder reemplazar a su abstracción sin romper el programa.

Si una clase promete comportarse como `EmpresaService`, cualquier implementación debería respetar ese contrato.

Si pides "un cargador USB-C", cualquier cargador compatible debería funcionar. Si uno tiene forma USB-C pero quema el equipo, no respeta el contrato.

En Spring Boot, si tienes:

```
private final EmpresaService service;
```

El controller no debería depender de detalles internos de `EmpresaServiceImpl`. Solo necesita que el contrato de `EmpresaService` se cumpla.

I: Interface Segregation Principle

Una clase no debería estar obligada a implementar métodos que no necesita.

Si una interfaz crece demasiado, las implementaciones terminan con métodos vacíos, falsos o con `return null`.

En tu service aparece `obtenerEmpresaXNumDoc` retornando `null`. Para aprender `assertNull` está bien, pero en diseño real puede ser señal de contrato incompleto o método pendiente.

Si una interfaz tiene demasiadas responsabilidades, puedes separarla:

```
public interface EmpresaQueryService {  
    ResponseEntity<BaseResponse<Empresa>> obtenerEmpresa(Long id);  
}  
  
public interface EmpresaCommandService {  
    ResponseEntity<BaseResponse<Empresa>> crear(EmpresaRequest request);  
    ResponseEntity<BaseResponse<Empresa>> actualizar(Long id, EmpresaRequest request);  
}
```

D: Dependency Inversion Principle

Las clases de alto nivel deben depender de abstracciones, no de detalles concretos.

Tu controller depende de `EmpresaService`, no directamente de `EmpresaServiceImpl`. Eso permite reemplazar la implementación, probar con mocks y mantener bajo el acoplamiento.

Un conductor no debería depender de una marca específica de motor para manejar. Depende de una interfaz: volante, acelerador, freno. El motor concreto puede cambiar.

```
@RestController
@RequiredArgsConstructor
public class EmpresaController {
    private final EmpresaService service;
}
```

Esta idea también hace posible tus tests de controller: reemplazas `EmpresaService` por un mock con `@MockBean`.

8. Cómo Pensar Un Test Nuevo

Antes de escribir código, escribe la historia del test en una frase.

Pregunta	Ejemplo
¿Qué quiero probar?	Crear empresa cuando no existe.
¿Qué condición preparo?	El repository dice que el documento no existe.
¿Qué acción ejecuto?	Llamo a <code>empresaService.crear(request)</code> .
¿Qué resultado espero?	Código OK, objeto presente y llamada a <code>save</code> .

Plantilla Recomendada

```
@Test
@DisplayName("Deberia hacer X cuando ocurre Y")
void deberiaHacerXCuandoOcurreY() {
    // Arrange
    // datos y mocks

    // Act
    // metodo real

    // Assert
    // verificaciones
}
```

Checklist Mental

- ¿El nombre del test cuenta una historia?
- ¿El Arrange prepara solo lo necesario?
- ¿El Act tiene una acción principal?
- ¿El Assert verifica comportamiento, no detalles irrelevantes?
- ¿El test falla si la lógica real se rompe?
- ¿El test no depende de PostgreSQL si es unit test?

9. Mini Diccionario De Testing

Término	Explicación simple
JUnit	Framework para escribir y ejecutar tests en Java.
Mockito	Librería para crear mocks y controlar dependencias falsas.
Mock	Objeto falso que simula una dependencia.
Stub	Respuesta preparada de un mock.
Assertion	Verificación de que algo ocurrió como esperabas.
@Test	Marca un método como prueba ejecutable.
@BeforeEach	Ejecuta preparación antes de cada test.
@WebMvcTest	Levanta solo la capa web para probar controllers.
@SpringBootTest	Levanta un contexto más completo de Spring.

10. Resumen Final

Si entiendes esta frase, entiendes la base de la clase:

Un buen test prepara un escenario, ejecuta una acción y verifica una consecuencia observable.

En tu proyecto, `EmpresaServiceImplTest` es el centro para entender unit testing y Mockito. `EmpresaControllerTest` es el ejemplo para entender pruebas web con Spring Boot. SOLID aparece por debajo como diseño: separar responsabilidades, depender de interfaces y escribir clases pequeñas hace que testear sea mucho más fácil.

Concepto	Frase para recordarlo
Unit Test	Prueba una pieza pequeña, con el mundo controlado.
Mockito	Crea actores falsos para controlar dependencias.
AAA	Preparo, ejecuto, verifico.
SOLID	Diseña piezas pequeñas, reemplazables y fáciles de probar.
Spring Boot	Organiza la app en capas: controller, service, repository.

Guía generada para estudiar sobre el proyecto local: C:\Users\OS\Documents\New project 9\unit-testing.